VISIONINSPECT-AI

Manufacturing Defect Detection & Quality Inspection System

Chamarthy Lakshyatha
Backend Development

Table of Contents

1. MODULE PURPOSE ---------- 3
2. PROJECT OBJECTIVE ---------- 4
3. BACKEND OBJECTIVES ---------- 4
4. TECHNOLOGY STACK ---------- 4
5. BACKEND FOLDER STRUCTURE ---------- 5
6. MAIN RESPONSIBILITIES ---------- 6
7. CORRECT MAIN BACKEND FLOW ---------- 7
8. APPLICATION – app.py ---------- 8
9. CONFIGURATION – config.py ---------- 8
10. AUTHENTICATION ---------- 9
11. USER ROLES ---------- 10
12. IMAGE UPLOAD ---------- 10
13. IMAGE VALIDATION ---------- 11
14. INSPECTION APIs ---------- 11
15. COMPLETE PREDICTION WORKFLOW ---------- 12
16. AI INFERENCE INTEGRATION ---------- 13
17. HEATMAP GENERATION ---------- 13
18. SEVERITY SCORING ---------- 14
19. QUALITY-CONTROL DECISION ---------- 14
20. DATABASE DESIGN ---------- 14
21. DATABASE PERSISTENCE ---------- 15
22. INSPECTION HISTORY ---------- 15
23. ANALYTICS ---------- 16
24. REPORTS AND EXPORT ---------- 17
25. DATASET APIs ---------- 17
26. FRONTEND–BACKEND INTEGRATION ---------- 17
27. CORS ---------- 19
28. SECURITY ---------- 19
29. ERROR HANDLING ---------- 19
30. TESTING AND VALIDATION ---------- 19
31. COMPLETE END-TO-END DATA FLOW ---------- 20
32. INPUTS AND OUTPUTS ---------- 21
33. INTEGRATION WITH OTHER PROJECT MODULES ---------- 22
34. BACKEND METHODOLOGY ---------- 22
35. CURRENT IMPLEMENTATION BOUNDARIES ---------- 23
36. TEST RESULT ---------- 24
37. CONCLUSION ---------- 24

1. MODULE PURPOSE

The Backend module acts as the central integration layer between the **frontend, AI inference pipeline, processing services, and database**.

It provides REST APIs for:

- User registration and login
- JWT authentication
- Role-based authorization
- Image upload
- Image validation
- AI inference integration
- Heatmap generation
- Severity scoring
- Quality-control decisions
- Inspection persistence
- Inspection history
- Analytics
- JSON/CSV report export

Overall Workflow

Authentication
↓
Image Upload
↓
Image Validation
↓
AI Inference
↓
Heatmap Generation
↓
Severity Scoring
↓
Quality Decision
↓
SQLite Persistence
↓
History
↓
Analytics
↓
JSON/CSV Export
↓
Frontend Dashboard

2. PROJECT OBJECTIVE

VisionInspect-AI is designed to support automated product-quality inspection using computer-vision-based analysis.

The backend provides the infrastructure required to connect the frontend with the AI inspection workflow and persist inspection results.

Instead of keeping AI results temporarily, the backend stores inspection information so it can be used for:

- Inspection history
- Analytics
- Quality decisions
- Reports
- Dashboard visualization

---

3. BACKEND OBJECTIVES

The backend is responsible for:

- Providing REST APIs
- Registering and authenticating users
- Generating and validating JWT tokens
- Enforcing authorization
- Receiving product images
- Validating uploaded files
- Integrating with AI inference
- Processing anomaly information
- Generating heatmaps
- Calculating severity
- Generating quality-control decisions
- Persisting inspection information
- Providing history
- Providing analytics
- Providing JSON/CSV export
- Supporting automated testing

---

4. TECHNOLOGY STACK

Technology	Purpose
Python	Backend programming
Flask	REST API framework
Flask Blueprints	Modular API organization
SQLite	Persistent database
PyJWT	JWT authentication

Technology	Purpose
Werkzeug	Password hashing and secure filename handling
Pillow	Image validation
OpenCV	Heatmap generation
NumPy	Numerical/image-array processing
Pytest	Automated testing
Flask Test Client	API testing

---

5. BACKEND FOLDER STRUCTURE

```
BACKEND/
└── backend/
    ├── __init__.py
    ├── app.py
    ├── config.py
    ├── bootstrap.py
    ├── database_utils.py
    ├── auth.py
    ├── create_supervisor.py
    ├── debug_auth.py
    │
    ├── database/
    │   ├── connection.py
    │   └── models.py
    │
    ├── models/
    │   ├── paper.py
    │   └── user.py
    │
    ├── routes/
    │   ├── __init__.py
    │   ├── auth.py
    │   ├── upload.py
    │   ├── inspection.py
    │   ├── history.py
    │   ├── analytics.py
    │   ├── dataset.py
    │   ├── reports.py
    │   ├── research.py
```

```
│   └── utils.py
│
├── services/
│   ├── __init__.py
│   ├── image_processing.py
│   ├── inference.py
│   ├── heatmap.py
│   ├── severity_scoring.py
│   ├── quality_control.py
│   └── dataset.py
│
├── utils/
│   └── helpers.py
│
├── tests/
│   ├── conftest.py
│   ├── test_backend.py
│   ├── test_defects_persistence.py
│   └── tmp_test_dbs/
│
├── uploads/
│
├── outputs/
│   └── heatmaps/
│
└── instance/
```

---

6. MAIN RESPONSIBILITIES

Folder/File	Responsibility
app.py	Creates Flask application and registers active routes
config.py	Stores application configuration
bootstrap.py	Backend initialization/bootstrap functionality
database_utils.py	SQLite database initialization, path resolution and connections
routes/	REST API endpoints
services/	Image, AI, heatmap, severity, quality-control and dataset logic
database/	Additional database connection/model helpers
models/	Auxiliary model definitions
utils/	General helper functions

Folder/File	Responsibility
tests/	Automated backend tests
uploads/	Uploaded images
outputs/heatmaps/	Generated heatmap images
instance/	Runtime/database instance data

The active Flask implementation mainly uses:

app.py
config.py
database_utils.py
routes/
services/
uploads/
outputs/
tests/
instance/

Some files such as routes/research.py, routes/reports.py, root-level auth.py, and files under models/ appear to be auxiliary/prototype modules because they are not registered in the active Flask application factory.

---

7. CORRECT MAIN BACKEND FLOW

Frontend
↓
app.py
↓
routes/inspection.py::predict_inspection
↓
Authentication and request validation
↓
Secure filename + UUID filename
↓
Image saved to UPLOAD_FOLDER
↓
services/inference.py::run_inference
↓
services/image_processing.py::preprocess_image
↓
Pillow image validation
↓
predict(image_path=processed_image, category=category)
↓

services/heatmap.py
↓
services/severity_scoring.py
↓
services/quality_control.py
↓
SQLite persistence
↓
JSON response

Important execution detail: the image is saved first inside predict_inspection(). Pillow validation occurs afterward inside preprocess_image().

---

8. APPLICATION – app.py

app.py is the main entry point of the Flask backend.

Responsibilities

- Creates the Flask application
- Loads configuration
- Enables CORS
- Creates required directories
- Initializes the database
- Registers active Blueprints
- Provides application-level endpoints
- Registers report aliases
- Serves generated heatmap files

---

9. CONFIGURATION – config.py

The backend uses centralized configuration.

Important configuration values include:

DATABASE_PATH
UPLOAD_FOLDER
HEATMAP_FOLDER
SECRET_KEY
MAX_CONTENT_LENGTH
ALLOWED_EXTENSIONS

These control:

- Database location
- Image storage
- Heatmap output
- Security configuration
- Maximum request size
- Allowed file extensions

10. AUTHENTICATION

The backend uses JWT-based authentication.

APIs

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

POST /api/auth/logout

Registration

Registration accepts:

- Username
- Email
- Password
- Optional role

A user can submit:

factory_supervisor

as the role.

There is **no separate admin approval flow**.

Registration Flow

Username + Email + Password + Optional Role

↓

Validation

↓

Password Hashing

↓

User Creation

↓

SQLite

Login Flow

Username + Password

↓

User Lookup

↓

Password Verification

↓

Role Retrieval

↓

JWT Generation

↓

Access Token

Protected APIs require:

Authorization: Bearer <access_token>

---

11. USER ROLES

The active backend contains two roles:

Quality Inspector

A Quality Inspector can:

- Register/login
- Upload images
- Perform inspections
- View permitted inspection history
- View inspection results

Factory Supervisor

A Factory Supervisor can:

- Register/login
- Upload images
- Perform inspections
- View inspection information
- Access analytics
- Access reports
- Export report data
- Access broader inspection information

Role Summary

Functionality	Quality Inspector	Factory Supervisor
Register/Login	✓	✓
Upload Images	✓	✓
Perform Inspection	✓	✓
View Permitted History	✓	✓
Analytics	✗	✓
Reports	✗	✓
JSON/CSV Export	✗	✓
Broader Inspection Visibility	✗	✓

Quality Engineer and Admin are not implemented in the active backend version.

---

12. IMAGE UPLOAD

API

POST /api/upload

Input

multipart/form-data

file = product image

The upload route performs extension validation, handles the uploaded file, and uses Werkzeug's secure_filename() for safe filename handling.

Upload Flow

Frontend
↓
Multipart Request
↓
Upload Route
↓
Extension Validation
↓
Secure Filename
↓
Image Storage
↓
Response

Important

POST /api/upload does **not** call Pillow validation.

UUID-prefixed filenames are used specifically in:

POST /api/inspection/predict

---

13. IMAGE VALIDATION

Image validation is handled through:

services/image_processing.py

The image-processing service is reached through:

services/inference.py

Pillow is used to verify image existence and readability during the prediction workflow.

Saved Image
↓
preprocess_image()
↓
Pillow Validation
↓
Validated Image
↓
AI Inference

---

14. INSPECTION APIs

GET /api/inspection

POST /api/inspection

GET /api/inspection/<id>

PUT /api/inspection/<id>

POST /api/inspection/predict

The main AI prediction endpoint is:

POST /api/inspection/predict

Input

- Product image
- Product category
- JWT access token

Output

The response can contain information such as:

- Inspection ID
- Filename
- Status
- Anomaly score
- Defect
- Confidence
- Severity
- Quality decision
- Recommended action
- Heatmap information

---

15. COMPLETE PREDICTION WORKFLOW

1. User sends image and category
2. JWT authentication is checked
3. Image field is checked
4. Category is checked
5. Filename and extension are checked
6. Filename is sanitized
7. UUID-prefixed filename is created
8. Image is saved
9. Image existence and readability are validated with Pillow
10. AI prediction is called
11. Heatmap is generated
12. Defective results receive severity scoring
13. Quality decision is calculated
14. Inspection data is saved to SQLite
15. Defects and quality decision are saved
16. Transaction is committed
17. JSON result is returned

---

16. AI INFERENCE INTEGRATION

The inference layer is implemented in:

services/inference.py

After preprocessing, the backend calls:

predict(image_path=processed_image, category=category)

when the model implementation is available.

The expected prediction information can include:

- Image name
- Category
- Status
- Anomaly score
- Anomaly map
- Prediction mask
- Defect
- Confidence

Important Implementation Boundary

The trained predict/model implementation is **not confirmed in the provided backend files**.

The backend contains a deterministic fallback for demonstration/testing when the model cannot be imported.

The fallback should **not** be described as the trained AI model.

---

17. HEATMAP GENERATION

The heatmap service converts anomaly information into a visual representation.

Methodology

Anomaly Map

↓

Float Conversion

↓

Gaussian Blurring

↓

Percentile Clipping

↓

Normalization

↓

Gamma Adjustment

↓

Thresholding

↓

Resizing

↓

OpenCV Color Mapping

↓

Alpha Blending

↓

Heatmap Image

The heatmap helps visualize where anomaly evidence is concentrated.

---

18. SEVERITY SCORING

The severity-scoring module converts inspection evidence into a severity score.

Formula

Severity =

0.30 × Area

+ 0.25 × Location

+ 0.25 × Defect Type

+ 0.20 × Confidence

Severity Levels

80 and above → Critical

60–79.99 → High

40–59.99 → Medium

Below 40 → Low

The purpose is to convert AI evidence into an interpretable quality-severity value.

---

19. QUALITY-CONTROL DECISION

The quality-control service converts inspection information into an operational decision.

Condition	Decision	Recommended Action	Result
Normal	Accept	Ready for Dispatch	PASS
Defective ≥ 80	Reject	Scrap Product Immediately	FAIL
Defective ≥ 60	Reject	Reject Product	FAIL
Defective ≥ 40	Manual Inspection	Send for Manual QC	PENDING REVIEW
Defective < 40	Rework	Send for Rework	FAIL

---

20. DATABASE DESIGN

The backend uses **SQLite** for persistent storage.

Main Tables

users

inspections

inspection_results

defects

quality_decisions

reports

Relationship

users
 |
 | 1 : Many
 ↓
inspections
 |
 ├── inspection_results
 ├── defects
 └── quality_decisions

Reports Table

The reports table exists in the active database schema.

It stores aggregate report information such as:

- Total inspections
- Pass count
- Fail count
- Rework count
- Other aggregate report fields

It is **not a PDF-report storage table**.

---

21. DATABASE PERSISTENCE

After AI processing:

Inspection
 ↓
inspections
 ↓
inspection_results
 ↓
defects
 ↓
quality_decisions

Database operations use transactions.

Successful operations are committed:

COMMIT

If a database error occurs:

ROLLBACK

This helps maintain consistency between related inspection records.

---

22. INSPECTION HISTORY

APIs

GET /api/history
GET /api/history/<id>
The History module retrieves previously stored inspection information.
It supports functionality such as:

- Inspection ownership
- Pagination
- Search
- Status filtering
- Category filtering
- Date filtering
- Individual inspection lookup
- Defect information
- Quality decision information

---

23. ANALYTICS

APIs

GET /api/analytics
GET /api/analytics/by-status
GET /api/analytics/reports
GET /api/analytics/export
Analytics can provide:

- Total inspections
- Average anomaly score
- Minimum score
- Maximum score
- Status distribution
- Category distribution
- Severity distribution
- Quality-decision distribution
- Category-level statistics

SQL operations include:
COUNT
AVG
MIN
MAX
GROUP BY
ORDER BY
Analytics uses persisted inspection data rather than rerunning the AI model for every analytics request.

---

24. REPORTS AND EXPORT

The active report endpoints are:

GET /api/analytics/reports

GET /api/analytics/export

GET /api/reports

GET /api/reports/export

The /api/reports paths are aliases registered in app.py, while their implementation comes from the analytics functionality.

Supported Functionality

- Report data retrieval
- JSON output
- CSV export
- Filtered report information
- Downloadable CSV data

routes/reports.py exists as an auxiliary module but is **not an active registered Blueprint**.

Therefore, **PDF inspection-report generation is not an active implemented backend feature.**

---

25. DATASET APIs

GET /api/dataset

GET /api/dataset/<category>/files

These APIs provide information about available dataset categories and files.

The dataset routes do **not explicitly require authentication**, so they should not be described as role-protected APIs.

---

26. FRONTEND–BACKEND INTEGRATION

Login

Frontend

↓

POST /api/auth/login

↓

Backend

↓

JWT Response

↓

Frontend

Inspection

Frontend

↓

POST /api/inspection/predict

↓

Backend

↓

AI Inference

↓

Database

↓

JSON Response

↓

Frontend Dashboard

History

Frontend

↓

GET /api/history

↓

Database

↓

JSON Response

↓

Frontend

Analytics

Frontend

↓

GET /api/analytics

↓

SQL Aggregation

↓

JSON Response

↓

Dashboard

Reports

Frontend

↓

GET /api/analytics/export

OR

GET /api/reports/export

↓

Stored Inspection Data

↓

JSON / CSV

---

27. CORS

The backend enables Cross-Origin Resource Sharing using:

CORS(app)

This allows frontend-backend communication when the applications run separately.

CORS is enabled broadly and is **not restricted to a configured frontend origin**.

---

28. SECURITY

The backend uses:

JWT Authentication

Authenticates protected API requests.

Password Hashing

Passwords are securely hashed before storage.

Role-Based Authorization

Controls access based on the authenticated user's role.

Ownership Filtering

Helps restrict inspection information according to user access.

Secure Filename Handling

Werkzeug's secure_filename() is used for uploaded filenames.

File Validation

Uploaded images are validated before AI processing.

Upload Size Limit

Flask's MAX_CONTENT_LENGTH limits the maximum request size.

---

29. ERROR HANDLING

The backend can return:

400 → Invalid request

401 → Authentication required/invalid

403 → Insufficient permissions

404 → Resource not found

413 → Request too large

500 → Internal server error

A 413 response is possible because of Flask's MAX_CONTENT_LENGTH.

There is **no custom 413 error handler**.

---

30. TESTING AND VALIDATION

Testing is performed using:

- Pytest
- Flask Test Client

Testing covers functionality including:

- Health check
- Registration
- Login
- Authentication
- Role handling
- Inspection history
- Analytics
- Supervisor access
- JSON export
- CSV export
- Multipart inspection
- Prediction
- Heatmap response
- Inspection IDs
- Defect persistence
- Quality-decision persistence

Test Command

python -m pytest backend/tests -q

Latest Result

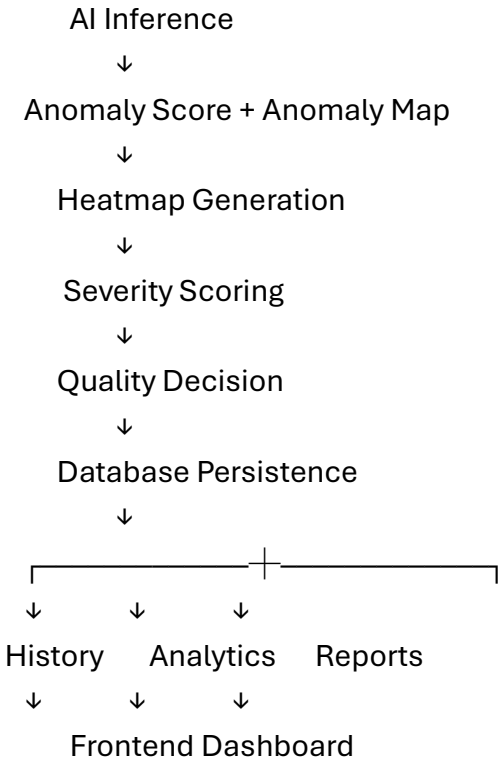
10 tests passed

Execution time can vary between runs.

---

31. COMPLETE END-TO-END DATA FLOW

USER
↓
Login / Register
↓
JWT Token
↓
Product Image
↓
REST API Request
↓
Authentication Check
↓
Authorization Check
↓
Image Validation
↓
File Storage
↓



---

32. INPUTS AND OUTPUTS

Inputs

- Username
- Password
- JWT access token
- Product image
- Product category
- Inspection information
- Search/filter parameters

Outputs

- JWT access token
- User information
- Inspection ID
- Inspection status
- Anomaly score
- Defect information
- Confidence
- Heatmap
- Severity score
- Severity level
- Quality decision
- Recommended action
- Inspection history

- Analytics
- JSON report data
- CSV exports

---

33. INTEGRATION WITH OTHER PROJECT MODULES

Frontend

Sends REST API requests and displays inspection, history, analytics, and reporting information.

AI Inference

The backend integrates with the AI inference layer and processes returned anomaly information.

Image Processing

Validates images before they enter the inspection pipeline.

Heatmap

Converts anomaly information into a visual representation.

Severity Scoring

Converts inspection evidence into a severity score.

Quality Control

Converts inspection results into an operational quality decision.

Database

Persists inspection and related information in SQLite.

Analytics and Reporting

Uses persisted inspection data to generate statistics and JSON/CSV exports.

Therefore, the backend acts as the **central communication and integration layer between the major project modules**.

---

34. BACKEND METHODOLOGY

Step 1 – Authentication

Authenticate the user and generate a JWT.

Step 2 – Request Validation

Validate authentication, image field, category, filename, and extension.

Step 3 – Secure File Handling

Sanitize the filename and generate a UUID-prefixed filename.

Step 4 – Image Storage

Save the image into the configured UPLOAD_FOLDER.

Step 5 – Image Validation

preprocess_image() uses Pillow to verify that the saved image exists and is readable.

Step 6 – AI Inference

Pass the validated image to the AI inference integration layer.

The inference service calls:

predict(image_path=processed_image, category=category)

Step 7 – Anomaly Analysis

Receive anomaly information from the prediction layer.

Step 8 – Heatmap Generation

Generate a visual heatmap from anomaly information.

Step 9 – Severity Scoring

Calculate the severity of a defective result.

Step 10 – Quality Decision

Generate the corresponding quality-control decision.

Step 11 – Persistence

Store inspection, result, defect, and quality-decision information in SQLite.

Step 12 – Transaction Commit

Commit the database transaction after successful persistence.

Step 13 – Response

Return the inspection result as JSON to the frontend.

---

35. CURRENT IMPLEMENTATION BOUNDARIES

1. Active roles are quality_inspector and factory_supervisor.
2. Quality Engineer and Admin are not implemented.
3. Registration accepts an optional role, including factory_supervisor.
4. There is no separate admin approval workflow.
5. POST /api/inspection/predict requires authentication but does not explicitly restrict the request to one specific role.
6. POST /api/upload performs extension validation and saves the file but does not call Pillow validation.
7. services/image_processing.py is reached through services/inference.py.
8. UUID-prefixed filenames are used specifically by /api/inspection/predict.
9. The backend calls predict(image_path=processed_image, category=category) when the model implementation is available.
10. The trained model implementation is not confirmed in the provided backend files.
11. The fallback inference is deterministic and intended for demonstration/testing.
12. SQLite is the current database implementation.
13. The reports table stores aggregate report information.
14. routes/reports.py is an inactive auxiliary Blueprint.
15. /api/reports and /api/reports/export are active aliases defined in app.py.
16. Active reporting supports JSON/CSV data functionality.
17. PDF inspection-report generation is not implemented as an active feature.
18. Dataset routes do not explicitly require authentication.
19. CORS is broadly enabled using CORS(app).
20. A 413 response is possible through MAX_CONTENT_LENGTH, but there is no custom 413 handler.

21. Personal authorship cannot be established from the source files alone.

---

36. TEST RESULT

$ python -m pytest backend/tests -q

10 tests passed

The latest backend test suite passed successfully. Execution time can vary between runs.

---

37. CONCLUSION

The VisionInspect-AI backend provides the **Flask API and integration workflow** for the complete inspection system.

It connects:

Authentication
↓
Image Validation
↓
AI Inference
↓
Heatmap Generation
↓
Severity Scoring
↓
Quality Decisions
↓
SQLite Persistence
↓
History
↓
Analytics
↓
JSON/CSV Export
↓
Frontend

The modular architecture separates API routes, processing services, database operations, configuration, and testing.

The backend therefore acts as the **central integration layer connecting the frontend, AI inference, processing services, and database into an API-driven quality-inspection workflow**.